

1. ¿Qué es una Base de Datos?

Una **base de datos** es:

Un conjunto de datos organizados y almacenados de forma estructurada para poder acceder a ellos fácilmente.

Ejemplos reales

- Usuarios de una app
 - Productos de una tienda
 - Alumnos de un instituto
-

2. Bases de Datos Relacionales

Las bases de datos relacionales organizan la información en **tablas**.

♦ Estructura

- **Tabla** → conjunto de datos
- **Fila (registro)** → un elemento
- **Columna (campo)** → una propiedad

Ejemplo

id	nombre	ciudad
1	Juan	Madrid
2	Ana	Sevilla

3. Sistemas Gestores de Bases de Datos (SGBD)

Son programas que permiten trabajar con bases de datos.

♦ Ejemplos

- MySQL

- PostgreSQL
 - Oracle
 - SQL Server
-

4. Primer contacto con SQL

SQL es el lenguaje para interactuar con bases de datos.

Crear una base de datos

```
CREATE DATABASE prueba;
```

Usar la base de datos

```
USE prueba;
```

Crear una tabla

```
CREATE TABLE usuarios (  
  id INT PRIMARY KEY,  
  nombre VARCHAR(50)  
);
```

Insertar datos

```
INSERT INTO usuarios VALUES (1, 'Juan');  
INSERT INTO usuarios VALUES (2, 'Ana');
```

Consultar datos

```
SELECT * FROM usuarios;
```

Modificar datos

```
UPDATE usuarios SET nombre = 'Pedro' WHERE id = 1;
```

Eliminar datos

DELETE FROM usuarios WHERE id = 2;

Mini práctica

1. Crear base de datos `empresa`
 2. Crear tabla `clientes` (id, nombre)
 3. Insertar 3 clientes
 4. Mostrar todos los clientes
 5. Cambiar un nombre
 6. Borrar un cliente
-

5. Introducción a JDBC

¿Qué es JDBC?

JDBC (Java Database Connectivity) es una API que permite a Java conectarse a bases de datos.

Traduce Java → SQL → Base de datos

Instalación

Descargar aquí: (Platform independent)

<https://dev.mysql.com/downloads/connector/j/>

En IntelliJ

1. Ve a **File** → **Project Structure**
2. En **Libraries**
3. Añadir (+)
4. Seleccionar el `.jar`
5. Aplicar

En Eclipse

1. Botón derecho en el proyecto
2. **Build Path** → **Add External JARs**

3. Seleccionas el `.jar`
-

6. Cómo funciona JDBC

Java → JDBC → Driver → Base de datos

- **JDBC** → interfaz común
 - **Driver** → adapta a cada BD
 - **Base de datos** → donde están los datos
-

7. Flujo básico en JDBC

1. Registrar driver
 2. Abrir conexión
 3. Ejecutar SQL
 4. Procesar resultados
 5. Cerrar conexión
-

8. Cadena de conexión

Ejemplo:

`jdbc:mysql://localhost:3306/prueba`

9. Primer ejemplo en Java

```
import java.sql.Connection;  
import java.sql.DriverManager;
```

```
public class Main {  
    public static void main(String[] args) throws Exception {  
  
        Connection c = DriverManager.getConnection(  
            "jdbc:mysql://localhost:3306/prueba",  
            "root",  
            "1234"  
        );  
    }  
}
```

```
        System.out.println("Conectado correctamente");  
        c.close();  
    }  
}
```

EJERCICIOS

1. Crear una base de datos llamada `tienda`.
2. Crear una tabla `productos (id, nombre)`.
3. Insertar 3 productos distintos.
4. Mostrar todos los productos con `SELECT *`.
5. Crear una tabla `clientes (id, nombre, ciudad)`.
6. Insertar 5 clientes con distintas ciudades.
7. Mostrar solo los clientes de una ciudad concreta.
8. Cambiar el nombre de un cliente.
9. Eliminar un cliente por id.
10. Crear una tabla `pedidos` relacionada con `clientes` (campo `cliente_id`) e insertar datos.
11. Crear dos tablas relacionadas:
 - a. `clientes (id, nombre)`
 - b. `pedidos (id, cliente_id, total)`
12. Insertar datos y mostrar pedidos con el nombre del cliente (JOIN).
13. Mostrar los clientes que **no tienen pedidos**.
14. Mostrar el **total gastado por cada cliente**.
15. Obtener el cliente que más ha gastado.
16. Crear una tabla `empleados` con:
 - a. id, nombre, salario
Insertar datos y mostrar:
 - b. salario máximo
 - c. salario mínimo
 - d. salario medio